MLOps Project Proposal

Intelligent Spending Pattern Detection for Firefly III Personal Finance Manager

Open Source Project: Firefly III (GitHub ≥ 2.5k stars)
Hosting Platform: Chameleon Cloud
Date: March 2026

4.2 Input Features and Output (1 pt) 11
4.2.1 Input Feature Vector 11
4.2.2 Output Specification 12
5. Serving Team Member 14
5.1 Operational Requirements (1 pt) 14
5.1.1 Request Volume and Latency 14
5.1.2 Justification 14
5.2 Model Output to Outcome Translation (1 pt) 14
5.2.1 From Scores to User-Facing Alerts 14
5.2.2 Outcome Tracking 15
6. Data Team Member 16
6.1 Data Flow Description (1 pt) 16
6.1.1 Data Arrival 16
6.1.2 Real-Time Processing for Inference 16
6.1.3 Training Data Processing 16
6.2 Training Data: Candidate Selection and Leakage Prevention (1 pt) 17
6.2.1 Candidate Selection 17
6.2.2 Avoiding Data Leakage 17
7. DevOps/Platform Team Member 18
7.1 Model Freshness Requirements (1 pt) 18
7.1.1 Retraining Frequency 18
7.1.2 Retraining Triggers and Conditions 18
7.1.3 Automation Lifecycle Integration 18
7.2 Scaling Requirements (1 pt) 19
7.2.1 Usage Profiles 19
7.2.2 Right-Sizing Strategy 19
7.2.3 Monitoring and Alerting 19
8. Requirements Checklist Compliance 21

1. Public-Facing Service Description

1.1 Service Overview

The public-facing service is a self-hosted personal finance management platform powered by Firefly III, an open-source application with over 16,000 GitHub stars. Firefly III enables users to track transactions, manage budgets, organize accounts, and generate financial reports through an intuitive web interface. It supports multi-currency tracking, recurring transactions, rule-based automation, and comprehensive reporting.

The service will be fully hosted on Chameleon Cloud infrastructure, running Firefly III in Docker containers with a PostgreSQL database backend. Users access the platform through a standard web browser to manage their day-to-day finances.

1.2 Target Audience

The anticipated user base consists of 10–50 concurrent users, primarily:

- **Privacy-conscious individuals:** Users who prefer self-hosted solutions over commercial platforms like Mint or YNAB, wanting full control over their financial data.
- **Budget-focused households:** Individuals and small families actively managing budgets and seeking insight into spending behavior.
- **University students and young professionals:** Users establishing financial habits who would benefit from automated spending analysis rather than manual review.
- **Small teams or co-living groups:** Shared households tracking communal expenses alongside personal budgets.

1.3 User Context

Users interact with Firefly III on a daily to weekly basis, logging transactions (manually or via automated bank imports), categorizing expenses, and reviewing financial summaries. The typical workflow involves entering transactions, checking dashboards, and occasionally adjusting budgets. The ML feature will complement this workflow by proactively surfacing spending anomalies that users might otherwise miss during their routine financial reviews.

The key pain point this addresses: users often do not realize their spending in a particular category (e.g., dining out, subscriptions, entertainment) has gradually crept upward until they do a manual end-of-month review. By then, the overspending has already occurred. The ML feature provides early, automated detection of these patterns.

2. Complementary ML Feature Design

2.1 Feature Overview

The ML feature is an Intelligent Spending Pattern Detector that identifies unusual spending behavior on a per-user, per-category basis. It operates as a complementary feature to Firefly III's existing transaction management: while Firefly III records and organizes financial data, the ML component analyzes that data to detect deviations from a user's established spending patterns.

This is not a standalone product—it is tightly integrated into Firefly III's existing dashboard and notification system, surfacing alerts like: "Your entertainment spending this month is 75% above your 3-month average" or "You've made 40% more transactions in the Shopping category this week compared to your norm."

2.2 Problem Framing

The task is framed as a per-user spending anomaly detection problem. Specifically, for each user and each spending category in each time period, the system must answer: "Is this user's current spending in this category significantly different from their historical baseline?"

This is a regression + threshold approach rather than a binary classification:

1. A model predicts the expected spending amount for each user-category-period combination based on historical features.
2. The actual spending is compared to the predicted expected spending.
3. Deviations exceeding a configurable threshold (e.g., more than 1.5 standard deviations) are flagged as anomalies.
4. Each anomaly receives a severity score (0–1), direction (over/under), and magnitude (percentage deviation).

2.3 Input Features

For each user + category + time window (the current rolling period, typically a month), the following features are extracted from Firefly III's transaction database:

2.3.1 Spending Amount Features

- Current period total spend in the category (absolute amount)
- Rolling mean of past 1, 3, and 6 months for the same category
- Rolling standard deviation over the past 3 and 6 months
- Current spend as a ratio of the rolling mean (deviation ratio)
- Current category spend as a proportion of total monthly spend (share of wallet)
- Historical share-of-wallet average for comparison

2.3.2 Transaction Frequency Features

- Number of transactions in the category this period
- Rolling mean transaction count for past 3 months

- Average transaction size this period vs. historical average (detecting fewer but larger purchases, or many small ones)
- Days since last transaction in this category

2.3.3 Temporal Features

- Month of year (integer 1–12, for capturing seasonality: holiday spending in December, back-to-school in August/September)
- Day-of-month spending velocity (cumulative spend at the current day vs. typical cumulative curve)
- Weekend vs. weekday transaction ratio for the category
- Is this month a known high-spend month (user-configurable: holidays, birthdays, etc.)

2.3.4 Cross-Category Features

- Total monthly spend across all categories (to distinguish category-specific anomalies from overall income changes)
- Number of categories with elevated spending (to detect if this is an isolated anomaly or a general pattern)
- Budget utilization percentage (if the user has set a budget for this category in Firefly III)

2.4 Model Output

For each user-category pair in the current period, the model outputs:

- **Anomaly score:** A continuous value from 0 to 1 indicating how unusual the current spending is (0 = perfectly normal, 1 = extreme outlier).
- **Direction:** Whether the anomaly is an overspend (+) or underspend (–).
- **Magnitude:** The percentage deviation from the predicted expected value (e.g., +65% above the predicted baseline).
- **Confidence:** A measure of how reliable the prediction is, based on the amount of historical data available for this user-category pair (new users/categories will have lower confidence).
- **Contributing factors:** The top 2–3 features driving the anomaly (e.g., “transaction count up 50%”, “average transaction size doubled”) derived from feature importance or SHAP values.

2.5 Model Architecture

2.5.1 Primary Model (Trained/Retrained)

The primary detection model is a Gradient Boosted Decision Tree ensemble (XGBoost) trained to predict “expected spending” for a user-category-period, where large residuals between predicted and actual indicate anomalies.

Why XGBoost over deep learning approaches:

- Tabular feature data with relatively few features (~20–30) is the sweet spot for gradient boosting.
- Highly interpretable: feature importances and SHAP values can explain why a particular spending pattern was flagged.

- Fast training and inference: suitable for periodic retraining on modest Chameleon hardware.
- Works well with limited data per user (can leverage cross-user patterns during initial training).

As an alternative/ensemble member, an Isolation Forest model will be trained alongside the XGBoost model. The Isolation Forest excels at detecting multivariate outliers—spending patterns that are unusual across multiple feature dimensions simultaneously—without requiring labeled anomaly data. The final anomaly score can be a weighted combination of both models' outputs.

2.5.2 Optional LLM Component (Out-of-Box, Not Retrained)

An out-of-the-box LLM (e.g., a small open-source model) can be used to generate natural-language explanations of detected anomalies, converting structured outputs into user-friendly notifications. For example: "Your dining out spending jumped significantly this month. This appears to be driven by several large transactions in the second week. You've also exceeded your dining budget by 30%."

Per the project requirements, this LLM component is supplementary—the core trained/retrained model is the XGBoost + Isolation Forest ensemble.

2.6 Feedback Mechanism

The feedback loop is critical for model improvement and is a strict requirement. Here is the detailed design:

2.6.1 Explicit Feedback

When the system surfaces an anomaly alert in the Firefly III dashboard or notification system, the user can:

5. Mark as "Helpful" — confirms this was a genuinely useful alert (positive label).
6. Mark as "Not Useful" — the alert was technically correct but not actionable or interesting to the user.
7. Mark as "Expected" — the spending was unusual but planned (vacation, large purchase, gift). The user can optionally add a reason tag (e.g., "holiday", "one-time purchase").
8. Dismiss — user saw the alert but chose not to provide feedback (treated as weak negative signal).

2.6.2 Implicit Feedback

- If a user adjusts their budget for a category after seeing an alert, this is a positive signal (the alert drove useful action).
- If a user creates a Firefly III rule to recategorize transactions after an alert, the model can learn from the recategorization.
- Click-through rate on alerts: alerts that are consistently ignored may indicate low value.

2.6.3 Feedback Storage

All feedback is stored in a dedicated feedback table in the PostgreSQL database with the following schema: user_id, category, period, anomaly_score, feedback_type

(helpful/not_useful/expected/dismissed), feedback_timestamp, optional_reason_tag, and the feature vector at the time of prediction. This preserves the full context needed for retraining.

2.7 Retraining Strategy

The model will be retrained using the following approach:

2.7.1 Retraining Triggers

- Scheduled: Monthly retraining for each active user, coinciding with the end of each calendar month when a full month of new transaction data is available.
- Threshold-based: If a user provides 10 or more feedback signals since the last retraining, trigger an ad-hoc retraining for that user.
- Cold-start: When a new user has accumulated at least 2 months of transaction history, the system initiates the first personalized model training.

2.7.2 Retraining Process

9. Extract all historical features for the user from the feature store.
10. Incorporate feedback labels: transactions marked “Expected” adjust the target variable (that period’s spending becomes the new expected baseline). “Helpful” confirmations reinforce the current model. “Not Useful” signals are used to raise the anomaly threshold for that user-category pair.
11. Retrain the XGBoost model on the updated dataset. For users with limited data, a transfer learning approach is used: start from a global model trained across all users and fine-tune on the individual’s data.
12. Evaluate the retrained model against a held-out validation set (most recent month’s data). If performance degrades (measured by precision@k on known anomalies), roll back to the previous model version.
13. Deploy the new model for the user and log the model version, training data hash, and performance metrics.

3. External Dataset

3.1 Primary Dataset: Czech Financial Dataset (PKDD'99)

The primary external dataset is the Czech Financial Dataset, originally provided by a Czech bank for the PKDD'99 Discovery Challenge (a well-known data mining competition). This is a real-world, non-synthetic dataset with well-documented lineage.

3.1.1 Dataset Lineage

- **Who collected it:** Petr Berka and Marta Sochorová from the Laboratory of Intelligent Systems, Prague University of Economics, in cooperation with a Czech commercial bank.
- **How it was collected:** Extracted from the bank's production database systems, anonymized to remove personally identifiable information, and released for academic research.
- **Why it was collected:** Created for the PKDD 1999 Discovery Challenge to foster research in financial data mining. It has been widely cited in academic literature.
- **Format:** Relational database tables in CSV format covering accounts, transactions, loans, credit cards, and demographic data.

3.1.2 Dataset Contents

The dataset contains approximately 1,056,320 transaction records across 4,500 bank accounts, spanning from January 1993 to December 1998. Key tables include:

Table	Records	Description
trans	1,056,320	Individual transactions with date, type, amount, balance, category, and bank info
account	4,500	Account creation dates and associated branch districts
order	6,471	Permanent (standing) orders for payments
client	5,369	Client demographic information (birth date, gender, district)

3.1.3 Example Data Points

Below are representative examples from the transaction table (trans):

trans_id	account_id	date	type	amount	category
695247	2378	1998-01-05	WITHDRAWAL	2,452 CZK	HOUSEHOLD
695248	2378	1998-01-12	WITHDRAWAL	680 CZK	FOOD
695303	2378	1998-01-28	CREDIT	14,230 CZK	SALARY
710892	3104	1998-03-15	WITHDRAWAL	5,100 CZK	INSURANCE PAYMENT

3.2 Alignment with the Proposed Service

This dataset aligns with the Firefly III spending detection service in several important ways:

- It contains real financial transactions with categories, amounts, dates, and account identifiers—exactly the data structure that Firefly III generates.
- The multi-year span (1993–1998) allows the model to learn seasonal spending patterns and long-term trends.
- The category labels (household, food, insurance, etc.) map directly to Firefly III's category system.
- Having 4,500 accounts provides enough cross-user diversity to train a robust global baseline model that can be personalized per user.

To use this dataset for pre-training, the transaction categories will be mapped to common Firefly III categories (e.g., HOUSEHOLD → "Housing", FOOD → "Groceries"). The rolling statistical features (means, standard deviations, ratios) will be computed from this data to create a training set. This global model then serves as the starting point for per-user fine-tuning once users accumulate enough transaction history in Firefly III.

3.3 Supplementary Dataset: Kaggle Credit Card Transactions

As a secondary dataset, the "Credit Card Transactions Fraud Detection" dataset from Kaggle (by Kartik Shenoy) provides approximately 1.3 million synthetic but realistically modeled credit card transactions with merchant categories, amounts, and timestamps. While synthetic, it complements the Czech dataset by providing modern transaction categories (online shopping, streaming services, ride-sharing) that better reflect current spending patterns. This dataset will be used only for feature engineering validation and testing, not as the primary training data (since the project requires non-synthetic data).

4. Training Team Member

4.1 Model Type and Training/Retraining Approach (1 pt)

4.1.1 Primary Model: XGBoost Regression

The core model is an XGBoost gradient-boosted decision tree regressor. It is trained to predict the expected spending amount for a given user-category-period based on historical features. Anomalies are identified by the residual (difference) between predicted and actual spending.

Training approach:

14. Global pre-training: An initial global model is trained on the Czech Financial Dataset across all 4,500 accounts. This model captures general spending patterns shared across users (e.g., salary deposits are regular, utility payments are monthly, food spending is relatively stable).
15. Per-user fine-tuning: Once a Firefly III user has accumulated 2+ months of data, the global model is fine-tuned on the user's personal transaction history using transfer learning (the global model's weights serve as the starting point, then the model is updated with user-specific data with a lower learning rate).
16. Feedback-driven retraining: As users provide feedback on anomaly alerts, the model is retrained monthly. "Expected" labels adjust the target values (making previously anomalous spending the new normal). "Helpful" labels reinforce the current predictions. The model version, training data snapshot, and evaluation metrics are logged for reproducibility.

4.1.2 Secondary Model: Isolation Forest

An Isolation Forest is trained alongside the XGBoost model for multivariate outlier detection. Unlike XGBoost which predicts expected values, the Isolation Forest directly identifies data points that are "easy to isolate" (i.e., anomalous) in the feature space.

Training approach:

- Trained on the same feature vectors as XGBoost but without target labels (unsupervised).
- Retrained monthly alongside the XGBoost model using the same updated feature set.
- The contamination parameter (expected proportion of anomalies) is tuned using historical feedback data.

4.1.3 Ensemble Strategy

The final anomaly score is a weighted average of the XGBoost residual-based score and the Isolation Forest anomaly score. The weights are initially set to 0.6 (XGBoost) and 0.4 (Isolation Forest) and are adjusted based on which model's predictions receive more "Helpful" feedback.

4.2 Input Features and Output (1 pt)

4.2.1 Input Feature Vector

Each training/inference example is a feature vector for one user + one category + one time period (month). The complete feature specification is:

#	Feature Name	Type	Description
1	**current_spend**	float (USD)	Total spend in category this period
2	**rolling_mean_1m**	float (USD)	Mean spend in category, past 1 month
3	**rolling_mean_3m**	float (USD)	Mean spend in category, past 3 months
4	**rolling_mean_6m**	float (USD)	Mean spend in category, past 6 months
5	**rolling_std_3m**	float (USD)	Std deviation of spend, past 3 months
6	**rolling_std_6m**	float (USD)	Std deviation of spend, past 6 months
7	**deviation_ratio**	float	current_spend / rolling_mean_3m
8	**share_of_wallet**	float (0-1)	Category spend / total monthly spend
9	**hist_share_of_wallet**	float (0-1)	Historical avg share-of-wallet
10	**txn_count**	int	Number of transactions this period
11	**hist_txn_count_mean**	float	Avg transaction count, past 3 months
12	**avg_txn_size**	float (USD)	Mean transaction amount this period
13	**hist_avg_txn_size**	float (USD)	Historical avg transaction size
14	**days_since_last_txn**	int	Days since most recent transaction
15	**month_of_year**	int (1-12)	Calendar month (seasonality)
16	**spending_velocity**	float	Cumulative spend / expected at day N
17	**weekend_txn_ratio**	float (0-1)	Weekend txns / total txns this period
18	**total_monthly_spend**	float (USD)	Total spend across all categories
19	**elevated_category_count**	int	# categories above 1.2x their mean
20	**budget_utilization**	float (0-1+)	Spend / budget for this category

4.2.2 Output Specification

For each user-category-period input, the model produces the following output structure:

Output Field	Type	Description
anomaly_score	float (0-1)	Combined anomaly score from ensemble (0 = normal, 1 = extreme outlier)
predicted_spend	float (USD)	XGBoost's predicted expected spending amount for this period
actual_spend	float (USD)	The user's actual spending in this category/period
direction	enum (+/-)	Whether spending is above (+) or below (-) the predicted value
magnitude_pct	float (%)	Percentage deviation: (actual - predicted) / predicted × 100
confidence	float (0-1)	Reliability of prediction based on data volume (low for new users)
top_factors	list[str]	Top 2-3 features driving the anomaly (from SHAP values)
model_version	string	Version identifier for the model that produced this prediction

5. Serving Team Member

5.1 Operational Requirements (1 pt)

5.1.1 Request Volume and Latency

The spending anomaly detection system operates primarily as a batch inference service rather than a real-time API, which significantly relaxes latency requirements.

Metric	Specification
Primary inference mode	Batch: nightly scheduled run at 2:00 AM for all active users
Batch throughput	~50 users × ~15 categories/user = 750 predictions per batch run
Batch completion target	< 5 minutes for full batch (all users, all categories)
Per-prediction latency	< 200ms per user-category prediction (feature extraction + inference)
On-demand refresh	User-triggered “refresh insights”: < 3 seconds for all categories for one user
Requests per second (peak)	~2-5 req/s during on-demand usage (multiple users refreshing simultaneously)
Availability target	99% uptime (acceptable to have brief downtime during retraining windows)
Model loading time	< 2 seconds (XGBoost + Isolation Forest models loaded into memory at service start)

5.1.2 Justification

The batch-first approach is justified because spending anomalies are not time-critical—a notification surfaced in the morning about the previous day’s spending is equally useful as a real-time alert. The on-demand refresh (< 3 seconds) is provided for users who want immediate analysis after entering a batch of transactions. Given the small user base (10–50 users), even the peak load is modest and can be handled by a single Chameleon VM instance.

5.2 Model Output to Outcome Translation (1 pt)

5.2.1 From Scores to User-Facing Alerts

The raw model output (anomaly scores, magnitudes, contributing factors) must be translated into actionable user-facing outcomes within the Firefly III interface. The translation pipeline works as follows:

17. Thresholding: Only anomalies with a score above a user-configurable threshold (default: 0.6) are surfaced. This prevents alert fatigue from minor fluctuations.
18. Severity classification: Anomalies are bucketed into three severity tiers based on their score and magnitude:

- Low (score 0.6–0.75): Subtle deviations shown as informational badges on the dashboard.

- Medium (score 0.75–0.9): Highlighted cards on the dashboard with a summary.
- High (score > 0.9): Prominent alert banners and optional email/push notifications.

19. Natural language generation: For medium and high severity alerts, the system generates a human-readable summary using a template engine (or optionally an LLM). Example output: “Your Entertainment spending this month ($340) is 78% higher than your 3-month average ($191). This is mainly driven by 5 more transactions than usual, with the average transaction being $12 larger.”
20. Dashboard integration: Alerts appear in a new “Spending Insights” widget on the Firefly III dashboard, alongside existing budgets and account summaries. Each alert includes: the category name and icon, the anomaly magnitude as a percentage, a sparkline showing the past 6 months of spending in that category, and action buttons for feedback (Helpful / Not Useful / Expected).
21. Actionable suggestions: For high-severity alerts, the system suggests concrete actions: “Consider adjusting your Entertainment budget to $350 to match your recent pattern” or “Review your recent Entertainment transactions to identify one-time vs. recurring expenses.”

5.2.2 Outcome Tracking

The system tracks whether alerts lead to user actions (budget adjustments, rule creation, transaction recategorization) within 7 days. This outcome tracking feeds back into the model evaluation pipeline to assess real-world effectiveness beyond statistical accuracy.

6. Data Team Member

6.1 Data Flow Description (1 pt)

6.1.1 Data Arrival

Data enters the system through two primary channels:

22. User-entered transactions: Users manually log transactions through the Firefly III web interface. Each transaction includes a date, amount, category, source account, destination account, and optional tags/notes. This data is immediately written to the PostgreSQL database.
23. Automated imports: Firefly III supports CSV imports and API-based data ingestion from external bank feeds. Imported transactions go through Firefly III's rule engine for automatic categorization before being committed to the database.

6.1.2 Real-Time Processing for Inference

When the nightly batch job runs (or when a user triggers an on-demand refresh):

24. Query extraction: A SQL query retrieves all transactions for the target user within the relevant time windows (current month + past 6 months) from the Firefly III transactions table.
25. Feature computation: The raw transactions are aggregated into the 20-feature vector described in Section 4.2. This involves computing rolling means, standard deviations, ratios, and temporal features. The computation is done in Python using pandas/numpy against the extracted data.
26. Feature caching: Computed features are stored in a feature cache table (separate from the Firefly III schema) to avoid recomputation. The cache is invalidated when new transactions are added for a user.
27. Model inference: The feature vector is fed to the loaded XGBoost and Isolation Forest models, producing the anomaly output described in Section 4.2.2.
28. Result storage: Inference results are written to an anomaly_alerts table with the user_id, category, period, scores, and metadata. The Firefly III dashboard queries this table to display alerts.

6.1.3 Training Data Processing

For model training and retraining, the data pipeline differs:

29. Historical extraction: All historical transactions for the user (or all users for global training) are extracted and converted to monthly aggregates per category.
30. Feature engineering: The same 20-feature computation is applied to each historical month, creating a time series of feature vectors. For each month, the "target" variable is the actual spending amount for the next month (the model learns to predict future spending from historical patterns).
31. Label integration: Feedback labels from the feedback table are merged. Months where users marked alerts as "Expected" have their target values adjusted (the actual spending becomes the expected value for that context).
32. Data serialization: The processed training dataset is serialized to Parquet format for efficient storage and fast loading during training.

6.2 Training Data: Candidate Selection and Leakage Prevention (1 pt)

6.2.1 Candidate Selection

Training data candidates are selected using the following criteria:

- **Minimum history requirement:** Only user-category pairs with at least 3 months of transaction history are included in the training set. This ensures enough data to compute meaningful rolling statistics.
- **Active category filter:** Categories with fewer than 2 transactions per month on average are excluded (these are too sparse for pattern detection and would introduce noise).
- **Outlier review:** Extreme outliers (spending > 10x the category median) are flagged for manual review before inclusion. These may represent data entry errors or genuinely rare events that could distort the model.
- **Temporal completeness:** Months with incomplete data (e.g., the user joined mid-month) are excluded from training to avoid biasing the model with partial-month spending.
- **External data integration:** Czech Financial Dataset records are processed through the same feature pipeline. Categories are mapped to Firefly III equivalents. Only accounts with 6+ months of history are included from the external dataset.

6.2.2 Avoiding Data Leakage

Data leakage prevention is critical for this time-series prediction task. The following safeguards are implemented:

- **Strict temporal splitting:** Training, validation, and test sets are split chronologically, never randomly. For example, if a user has 12 months of data: months 1–8 are training, months 9–10 are validation, months 11–12 are test. No future data ever leaks into the training set.
- **Rolling window features only:** All features are computed using only data available up to (but not including) the prediction month. The rolling_mean_3m for March uses only December, January, and February data. There is no look-ahead in any feature computation.
- **Feedback label timing:** Feedback labels are only used in retraining if the feedback was provided before the prediction date of the new training example. A user's feedback on a March alert is only incorporated when retraining for April and beyond.
- **Cross-user isolation:** When training per-user models, data from other users is never included (except in the global pre-training phase, where the model is intentionally cross-user). Per-user fine-tuning uses only that user's data.
- **Feature computation isolation:** The feature engineering pipeline runs identically for training and inference. There is a single shared function that computes features from raw transactions, ensuring no discrepancy between training features and serving features (training-serving skew).
- **No target leakage:** The target variable (next month's actual spending) is never included in the feature set. Features like budget_utilization use only the budget as set at the start of the month, not any mid-month adjustments.

7. DevOps/Platform Team Member

7.1 Model Freshness Requirements (1 pt)

7.1.1 Retraining Frequency

The models require retraining on a monthly cadence, aligned with the natural rhythm of personal finance data:

Component	Retraining Frequency	Justification
Global baseline model	Quarterly (every 3 months)	Cross-user patterns evolve slowly; quarterly is sufficient to capture macro-level shifts
Per-user XGBoost	Monthly (1st of each month)	Monthly aligns with budget cycles and provides enough new data for meaningful updates
Per-user Isolation Forest	Monthly (alongside XGBoost)	Must stay synchronized with XGBoost to ensure ensemble consistency
Alert threshold tuning	Monthly (post-retraining)	Thresholds adjusted based on accumulated feedback since last tuning

7.1.2 Retraining Triggers and Conditions

Beyond the scheduled monthly retraining, the following conditions trigger ad-hoc retraining:

- **Feedback accumulation:** If a user provides 10+ feedback signals (Helpful/Not Useful/Expected) since the last retraining, an immediate retrain is triggered for that user. This ensures the model adapts quickly to explicitly communicated user preferences.
- **Performance degradation:** If the model's precision@5 (proportion of top-5 alerts marked "Helpful") drops below 40% over a rolling 2-month window, retraining is triggered with adjusted hyperparameters.
- **Data drift detection:** If the distribution of incoming features shifts significantly (measured by Population Stability Index > 0.2), a full retraining is initiated. This captures scenarios like a user getting a raise (systematic spending increase across all categories).

7.1.3 Automation Lifecycle Integration

The retraining process fits into the following automated pipeline:

33. Cron trigger: A cron job on the 1st of each month at 3:00 AM initiates the retraining pipeline.
34. Data extraction: Latest transactions and feedback are pulled from PostgreSQL.
35. Feature computation: Updated feature vectors are computed and cached.
36. Model training: XGBoost and Isolation Forest are retrained per user.

37. Evaluation: New model is tested against held-out data. If metrics pass, the model is promoted to production. If not, the previous model is retained and an alert is sent to the operations dashboard.
38. Model registry update: The new model artifact, training data hash, hyperparameters, and evaluation metrics are logged.
39. Rollout: The inference service loads the new model versions for subsequent batch runs.

7.2 Scaling Requirements (1 pt)

7.2.1 Usage Profiles

Scenario	Characteristics	Resource Requirements
Typical usage	Nightly batch: 50 users, 750 predictions. Sporadic on-demand: 2-3 users/hour	1 vCPU, 2 GB RAM for inference service. Batch completes in < 2 minutes
Peak usage	End of month: all 50 users trigger on-demand refresh + nightly batch. ~5 concurrent requests	2 vCPU, 4 GB RAM. Burst capacity for concurrent feature computation
Retraining	Monthly: 50 per-user models trained sequentially. ~10 minutes total	4 vCPU, 8 GB RAM during training window (can share with inference VM)
Global model training	Quarterly: Full Czech dataset + all user data. ~30 minutes	4 vCPU, 8 GB RAM. Can run overnight during low-usage window

7.2.2 Right-Sizing Strategy

Given the modest scale of this deployment (10–50 users on Chameleon), the right-sizing approach prioritizes cost efficiency over high availability:

- **Single-node deployment:** All components (Firefly III, PostgreSQL, ML inference service, training pipeline) run on a single Chameleon VM. A medium instance (4 vCPU, 8 GB RAM, 80 GB SSD) handles all workloads comfortably.
- **Resource sharing with time-based isolation:** Training jobs run at 3:00 AM when user activity is near zero. Inference serves batch results at 2:00 AM and on-demand requests during business hours. There is no contention because heavy workloads are scheduled during off-peak hours.
- **Vertical scaling path:** If usage grows beyond 50 users, the first step is upgrading to a larger VM (8 vCPU, 16 GB RAM) rather than adding horizontal complexity. The system does not require distributed computing for up to ~500 users.
- **Model size management:** Each per-user XGBoost model is approximately 50–200 KB. The Isolation Forest model is approximately 100–300 KB per user. For 50 users, total model storage is under 25 MB—negligible disk impact.
- **Database growth:** The anomaly_alerts and feedback tables grow linearly with users and time. At 50 users with 15 categories each and 12 months of history, the alert table has ~9,000 rows—trivial for PostgreSQL. The feature cache table is similarly small.

7.2.3 Monitoring and Alerting